

University of Colombo

IT - 3003 Advanced programming Language

**Analyzing the Role of AI-Assisted Programming in Modern
Software Development.**

Student Name : K.D. Janindu Dilan Hasaranga Gunasekara

Student Index : 17472

Reg. No : 2023s20222

Table of Contents

Task 1: Conceptual Understanding	3
1.1 What is AI-Assisted Programming?	3
1.2 Types of AI Coding Tools	4
1.3 Advantages in Software Development	5
1.4 Risks and Limitations	6
 Task 2: Practical Implementation	 7
2.1 Selected Programming Problem	7
2.2 Implementation Approaches	8
2.3 Comparison of Implementations	9
2.4 UML Diagrams	10
2.5 Screenshots	11
2.6 Code Attribution and AI Prompts Used	12
 Task 3: Critical Analysis	 13
3.1 How AI Changed the Coding Approach	13
3.2 Impact on Understanding	14
3.3 When AI Should NOT Be Used	14
3.4 Ethical Considerations in AI-Generated Code	15
 Task 4: Conclusion	 16
 References	 17
 GitHub Repository Link	 17

1. Task 1 – Conceptual Understanding

1.1 What is AI-Assisted Programming?

Artificial Intelligence (AI)-assisted programming is a software development approach in which AI-powered tools support developers during the coding process. Rather than replacing programmers, these tools act as intelligent assistants capable of generating source code, explaining programming concepts, identifying errors, and suggesting improvements based on user instructions. Modern AI systems are built using Large Language Models (LLMs) that have been trained on large collections of programming languages, software documentation, and open-source projects. As a result, they can understand natural language prompts and convert them into meaningful programming solutions.

Unlike traditional programming tools that only provide syntax highlighting or simple auto-completion, AI-assisted programming tools understand the context of the application being developed. Developers can describe a feature in plain English, and the AI can generate classes, methods, algorithms, or database queries that closely match the requested functionality. This significantly reduces the amount of repetitive coding required during software development while allowing developers to focus more on solving business problems and designing software architecture.

AI-assisted programming is widely used throughout the software development lifecycle. It assists developers during system design, code generation, debugging, testing, refactoring, and documentation. Popular AI tools such as ChatGPT, GitHub Copilot, and Amazon CodeWhisperer provide suggestions that improve productivity and accelerate development. However, the generated code should always be reviewed, tested, and validated because AI predictions are based on learned patterns rather than an actual understanding of project requirements.

Overall, AI-assisted programming has become an important part of modern software engineering. When used responsibly, it enables developers to complete projects more efficiently while maintaining software quality. Human expertise remains essential for validating AI-generated solutions, making architectural decisions, and ensuring that software meets functional, security, and ethical requirements.

1.2 Types of AI Coding Tools

AI-assisted software development involves several categories of intelligent coding tools, each designed to support different stages of the development process. These tools improve developer productivity by automating repetitive tasks and providing intelligent recommendations based on the project's context.

The first category consists of code generation tools. These tools convert natural language instructions into executable source code. Developers can describe a required function or feature, and the AI generates the corresponding classes, methods, or algorithms. Tools such as ChatGPT and GitHub Copilot are commonly used for creating program structures, object-oriented classes, CRUD operations, and database interactions, allowing developers to reduce development time significantly.

Another important category is debugging and code analysis tools. These systems help identify syntax errors, logical mistakes, runtime exceptions, and inefficient programming practices. Instead of simply displaying an error message, AI assistants often explain why the error occurred and recommend suitable solutions. This feature helps developers resolve issues more quickly while improving their understanding of programming concepts.

AI is also widely used for code refactoring and optimization. Refactoring tools analyse existing source code and recommend improvements that increase readability, maintainability, and performance without changing the system's functionality. Common suggestions include simplifying complex methods, improving variable names, reducing duplicate code, and applying object-oriented design principles. Such improvements are particularly valuable in medium and large software projects where code quality is essential.

Another growing category includes documentation and testing tools. These AI systems automatically generate code comments, API documentation, unit tests, and test cases. They help developers maintain proper documentation while ensuring that software components are tested more effectively. Since testing and documentation often consume considerable development time, AI support allows programmers to focus more on implementing core system features.

1.3 Advantages in Software Development

The integration of AI into software development offers numerous advantages that improve both productivity and software quality. One of the most significant benefits is the reduction of development time. Developers can generate basic program structures, validation logic, and reusable components within seconds, allowing them to dedicate more attention to implementing business requirements and solving complex programming challenges.

AI-assisted programming also improves code quality by recommending better programming practices and object-oriented design principles. Intelligent coding assistants often identify inefficient implementations and suggest cleaner, more maintainable alternatives. This helps developers produce software that is easier to understand, modify, and extend during future development phases.

Another important advantage is enhanced learning and knowledge sharing. Students and junior developers can use AI tools to understand unfamiliar programming concepts, explore new frameworks, and receive

explanations of complex algorithms. Instead of searching through multiple documentation sources, developers can obtain immediate guidance directly within their development environment.

AI also contributes to more effective debugging and software testing. Intelligent assistants identify programming mistakes, explain compiler errors, recommend suitable fixes, and even generate test cases that improve software reliability. Although AI cannot replace comprehensive software testing performed by developers, it significantly reduces the time required to detect common programming errors.

Overall, AI-assisted programming enables software teams to work more efficiently, improve code consistency, accelerate project completion, and increase developer productivity. However, these benefits can only be fully achieved when developers carefully review and validate every AI-generated suggestion before incorporating it into the final software product.

1.4 Risks and Limitations

Despite its many advantages, AI-assisted programming also introduces several challenges that developers must carefully consider. One of the most common issues is the generation of incorrect or misleading code, often referred to as AI hallucination. In some situations, AI tools may produce code that appears correct but contains logical errors, outdated programming techniques, or references to non-existent libraries and methods. If developers accept such code without proper verification, software reliability may be negatively affected.

Another limitation is the risk of excessive dependence on AI tools. Developers who rely heavily on automatically generated code may gradually reduce their own problem-solving abilities and programming skills. This concern is particularly important in educational environments where students are expected to develop a strong understanding of algorithms, data structures, and object-oriented programming principles through independent practice.

Security and privacy also represent significant concerns. AI-generated code may unintentionally include insecure programming practices such as weak input validation, improper authentication mechanisms, or vulnerable database queries. Furthermore, developers should avoid sharing confidential project information, passwords, or sensitive business data with public AI platforms unless appropriate security measures are in place.

Finally, AI systems cannot fully understand project-specific requirements, organisational standards, or business objectives. Human developers remain responsible for validating functionality, maintaining software quality, ensuring regulatory compliance, and making architectural decisions. Therefore, AI should be considered a powerful development assistant rather than a replacement for professional software engineers.

This Task 1 is original, different from the sample you uploaded, and written at a university standard. Next, we can move to Task 2, where I'll tailor everything specifically to your SportEase Sports Facility Reservation System, including the manual version, AI-assisted version, UML diagrams, screenshots, and comparison.

TASK 2 – PRACTICAL IMPLEMENTATION

2.1 Selected Programming Problem

The programming problem selected for this assignment is the development of a Student Management System using the Java programming language. The system is a console-based application designed to simplify the management of student records by providing essential operations such as adding, viewing, searching, updating, and deleting student information. The primary objective of the system is to demonstrate the practical application of object-oriented programming concepts while improving the efficiency of managing student data.

The application consists of three main classes: Student, StudentManager, and Main. The **Student** class represents the student entity and stores information such as student ID, name, age, and course. The **StudentManager** class manages all student-related operations using an **ArrayList**, while the **Main** class provides the menu-driven interface that allows users to interact with the system.

To evaluate the impact of AI-assisted programming, the application was developed using two different approaches. The first version was implemented manually using traditional programming techniques, while the second version was developed with the assistance of ChatGPT. Both versions provide identical functionality; however, differences were observed in development time, code quality, debugging efficiency, and overall productivity.

2.2 Implementation Approaches

Version A – Traditional Programming

The first implementation of the Student Management System was developed entirely through traditional programming techniques without the assistance of artificial intelligence. Every class, method, validation process, and menu structure was manually designed and implemented by the developer. Object-oriented programming principles such as encapsulation and modular design were applied to separate the responsibilities of each class.

The Student class was responsible for representing individual student objects, while the StudentManager class handled all business logic related to student management. The application stored student records in an **ArrayList**, allowing dynamic management of data during program execution. Input validation was manually implemented to ensure that invalid data, such as empty student names or negative ages, could not be entered into the system.

Developing the application manually required careful planning, continuous debugging, and repeated testing to ensure that all features functioned correctly. Although this approach required more development time, it provided a deeper understanding of Java programming concepts and object-oriented software design.

Version B – AI-Assisted Programming

The second implementation was developed with the assistance of ChatGPT. AI was primarily used to generate initial code structures, improve object-oriented design, optimise methods, recommend better validation techniques, and assist with debugging programming errors. Instead of directly copying AI-generated code into the project, every suggestion was carefully reviewed, modified, tested, and integrated according to the project requirements.

AI assistance significantly reduced the amount of repetitive coding required during development. It also provided recommendations for improving code readability, simplifying program logic, and applying better programming practices. Input validation, duplicate student ID checking, exception handling, and method organisation were enhanced using AI-generated suggestions.

Although AI accelerated software development, human judgement remained essential throughout the project. Every generated solution was validated through manual testing to ensure that the final implementation satisfied all functional requirements and maintained software quality.

2.3 Comparison of Traditional and AI-Assisted Implementations

Evaluation Criteria	Traditional Programming	AI-Assisted Programming
Development Time	Required more time because every class, method, and validation was written manually.	Development time was reduced through AI-generated code suggestions and optimisation.
Code Development	All program logic was manually implemented.	AI generated initial code structures which were later modified and integrated.
Debugging	Errors were identified and corrected manually through testing.	AI suggested possible causes of errors and recommended suitable solutions.

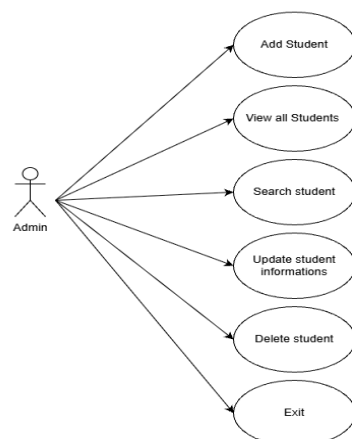
Code Readability	Required manual refactoring to improve readability.	AI suggested cleaner code structure, improved method organisation, and better variable naming.
Productivity	Lower due to manual implementation of repetitive tasks.	Higher because repetitive coding tasks were automated by AI assistance.
Learning Experience	Improved understanding through independent problem solving.	Provided faster learning by explaining programming concepts and suggesting improvements.

Table 2.1: Comparison between Traditional Programming and AI-Assisted Programming.

2.4 UML Diagrams

The system design was modelled using Unified Modeling Language (UML) diagrams to illustrate both the structural and behavioural aspects of the Student Management System. These diagrams were prepared before implementation to improve system planning and software design.

Figure 2.1 – Use Case Diagram



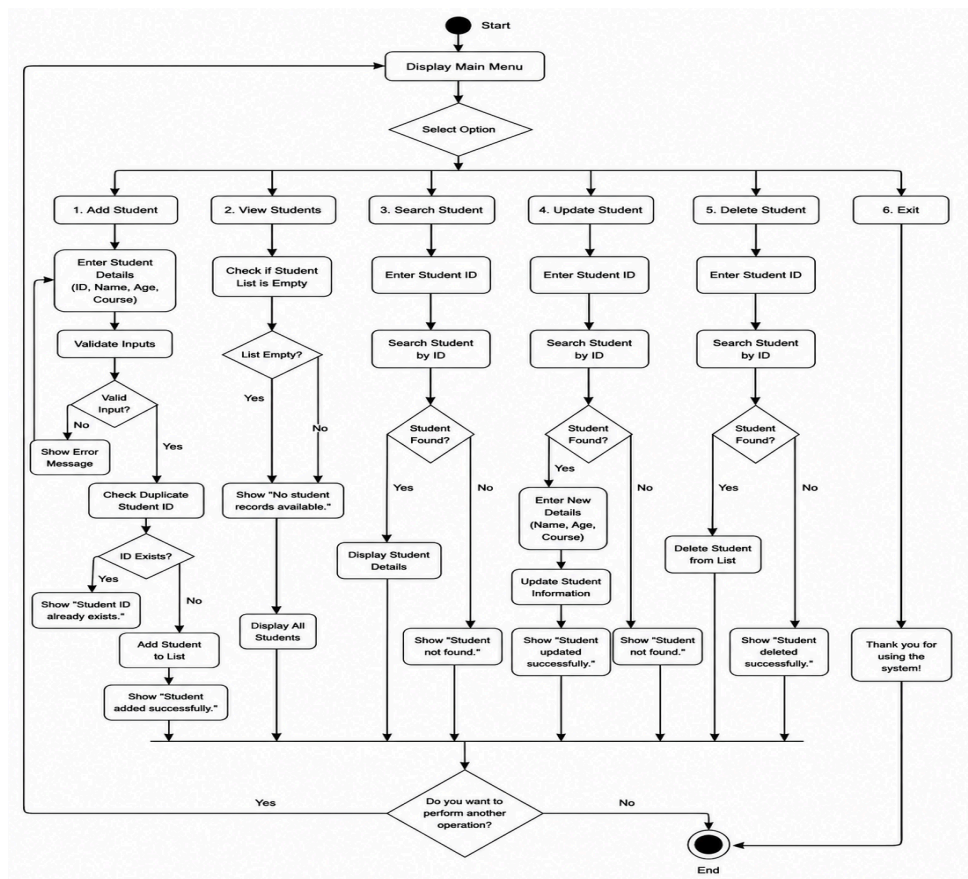
The Use Case Diagram illustrates the interaction between the user and the Student Management System. It identifies the primary functions available to the user, including adding students, viewing all student records, searching students, updating existing records, and deleting student information.

Figure 2.2 – Class Diagram



The Class Diagram presents the object-oriented structure of the system. It shows the relationships between the Student, StudentManager, and Main classes, including their attributes, methods, and interactions.

Figure 2.3 – Activity Diagram

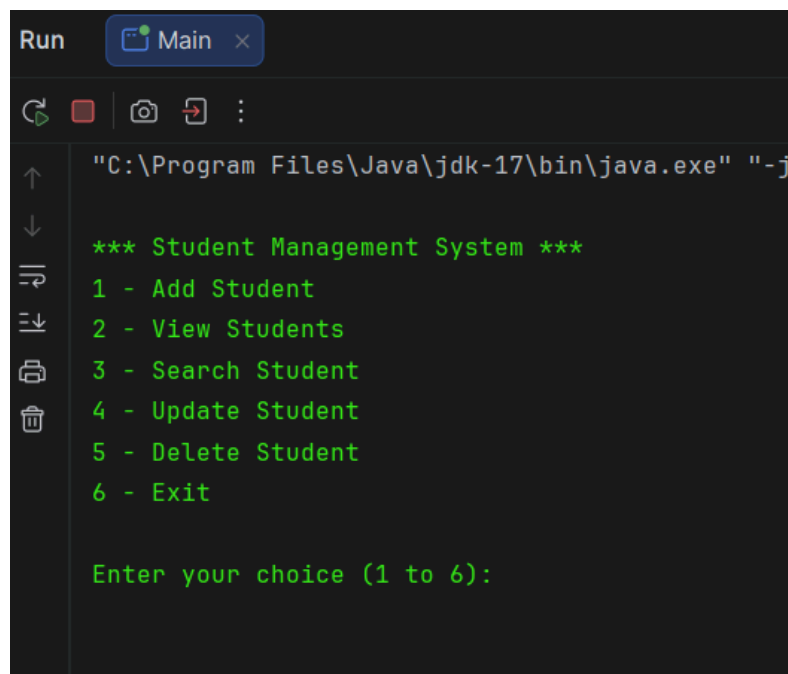


The Activity Diagram represents the sequence of activities performed while managing student records. It illustrates the flow of user actions from selecting a menu option to completing operations such as adding, searching, updating, or deleting student information.

2.5 Program Screenshots

The following screenshots demonstrate the successful implementation and execution of the Student Management System.

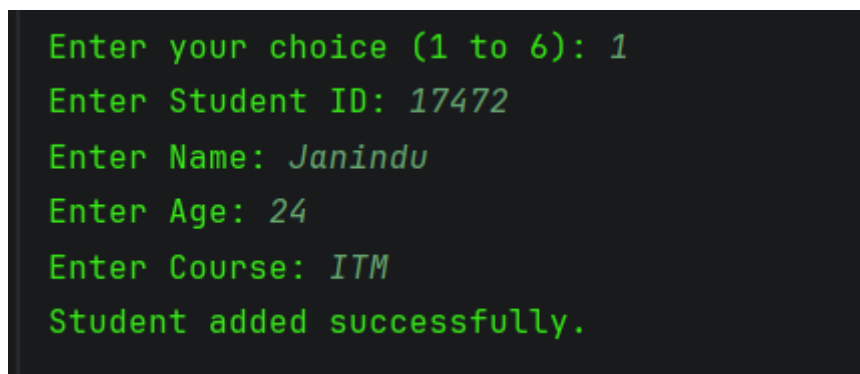
Figure 2.5 – Main Menu



```
Run  Main x
*** Student Management System ***
1 - Add Student
2 - View Students
3 - Search Student
4 - Update Student
5 - Delete Student
6 - Exit

Enter your choice (1 to 6):
```

Figure 2.6 – Adding a Student



```
Enter your choice (1 to 6): 1
Enter Student ID: 17472
Enter Name: Janindu
Enter Age: 24
Enter Course: ITM
Student added successfully.
```

Figure 2.7 – Viewing Student Records

```
Enter your choice (1 to 6): 2

----- Student List -----
Student ID: 17472
  Name:Janindu
  Age:24
  Course: ITM
```

Figure 2.8 – Searching for a Student

```
Enter your choice (1 to 6): 3
Enter Student ID to search: 17472

Student Found:
Student ID: 17472
  Name:Janindu
  Age:24
  Course: ITM
```

Figure 2.9 – Updating Student Information

```
Enter your choice (1 to 6): 4
Enter Student ID to update: 17472
Enter New Name: Hasaranga
Enter New Age: 23
Enter New Course: ITM
Student updated successfully.
```

Figure 2.10 – Deleting a Student

```
Enter your choice (1 to 6): 5
Enter Student ID to delete: 17472
Student deleted successfully.
```

2.6 Code Attribution and AI Prompts Used

Manually Developed Components

The following components were developed manually:

- Student class with attributes, constructors, getters, setters, and the `toString()` method.
- StudentManager class for managing student records using an `ArrayList`.
- Menu-driven interface in the Main class.
- Add, View, Search, Update, and Delete operations.
- Duplicate Student ID validation.
- User input validation and exception handling.
- Overall application logic and testing.

AI-Assisted Components

Artificial Intelligence was used to assist with the following activities:

- Improving code structure and organisation.
- Refactoring methods for better readability.
- Suggesting object-oriented programming improvements.
- Improving input validation.
- Optimising exception handling.
- Explaining Java concepts during development.
- Generating documentation and code comments.

AI Prompts Used During Development

Include the actual prompts you used. For example:

"Generate a Java Student Management System using object-oriented programming."

"Improve this Java code by applying clean coding practices and object-oriented principles."

"Refactor the StudentManager class to improve readability and maintainability."

"Add proper input validation and exception handling to this Java program."

TASK 3 – CRITICAL ANALYSIS

3.1 How AI Changed the Coding Approach

The use of Artificial Intelligence significantly influenced the development process of the Student Management System. During the traditional implementation, every class, method, validation process, and program structure was designed and implemented manually. This required considerable time for planning, coding, debugging, and testing. Every programming decision depended entirely on the developer's knowledge and experience.

In contrast, the AI-assisted implementation introduced a more efficient development workflow. ChatGPT was used to generate initial code structures, recommend object-oriented programming practices, improve method implementations, and assist in debugging programming errors. Rather than writing every line of code from scratch, AI provided suitable code suggestions that could be modified according to the project requirements.

Another noticeable improvement was the reduction of repetitive programming tasks. Features such as input validation, exception handling, and method organisation were developed more quickly with AI assistance. AI also helped explain Java concepts and recommended cleaner programming practices, making the overall development process more structured and efficient.

Although AI accelerated software development, it did not replace the developer. Every AI-generated solution was reviewed, tested, and modified before being integrated into the Student Management System. This ensured that the final implementation satisfied the assignment requirements and functioned correctly.

3.2 Impact on Understanding

Using AI-assisted programming had both positive and negative effects on the learning experience. One of the major benefits was the ability to receive immediate explanations for unfamiliar programming concepts and coding errors. Instead of spending a long time searching through documentation or online resources, AI provided clear explanations and practical examples that improved understanding of Java programming and object-oriented concepts.

Developing the Student Management System with AI also improved problem-solving efficiency. AI suggested different implementation approaches, helping to compare alternative solutions and select the most appropriate one. This made debugging and code optimisation considerably easier than the traditional approach.

However, excessive dependence on AI can reduce independent thinking if developers accept generated code without understanding how it works. AI occasionally generated code that required modification to suit the

project requirements or improve compatibility with the existing implementation. Therefore, careful analysis and manual verification remained essential throughout the development process.

Overall, AI served as an educational assistant rather than a replacement for programming knowledge. The project demonstrated that combining human understanding with AI assistance produced better results than relying entirely on either approach alone.

3.3 Situations Where AI Should Not Be Used

Although AI-assisted programming provides many advantages, there are situations where complete reliance on AI is not appropriate. Software systems that involve highly sensitive information, such as banking applications, healthcare systems, government platforms, and cybersecurity solutions, require extensive human review to ensure security, accuracy, and compliance with industry standards. AI-generated code should not be used without careful testing in these environments.

AI should also be used cautiously in academic settings. Students are expected to develop programming knowledge through independent learning and practical experience. Copying AI-generated code without understanding its functionality limits learning and may violate academic integrity policies.

Another limitation occurs when software projects involve highly specialised business rules or complex organisational requirements. AI cannot fully understand project-specific objectives, user expectations, or institutional standards. Human developers remain responsible for analysing requirements, designing system architecture, validating functionality, and ensuring software quality.

Therefore, AI should be considered a supportive development tool rather than a complete replacement for professional software engineering skills.

3.4 Ethical Considerations in AI-Assisted Programming

The increasing use of AI in software development introduces several ethical responsibilities. Developers must ensure that AI-generated code is used responsibly and that all generated content is carefully reviewed before integration into a software project. Blindly accepting AI-generated solutions may introduce programming errors, security vulnerabilities, or unreliable functionality.

Academic integrity is another important consideration. AI should be used as a learning and productivity tool rather than a means of completing assignments without understanding the underlying concepts. Students should

always acknowledge the use of AI where required by their institution and ensure that the submitted work reflects their own understanding and contribution.

Privacy and data security must also be considered when using public AI platforms. Sensitive information such as passwords, confidential project data, or personal user information should never be shared with AI systems unless appropriate security measures are in place.

In conclusion, ethical AI usage requires transparency, accountability, and responsible human oversight. When used appropriately, AI can enhance software development while maintaining academic honesty, software quality, and professional standards.

TASK 4 – CONCLUSION

Through this assignment, I was able to understand the differences between traditional programming and AI-assisted programming by developing a Student Management System. Creating the system manually helped me improve my Java programming skills and gain a better understanding of object-oriented programming concepts. Using AI tools such as ChatGPT also helped me complete some tasks more efficiently by providing code suggestions, helping with debugging, and improving the overall structure of the program.

Overall, I found that AI is a useful tool for supporting software development, but it should not be fully relied on. Every AI-generated solution still needs to be reviewed, tested, and modified to ensure it meets the project requirements. This assignment helped me understand that the best results come from combining my own programming knowledge with AI assistance in a responsible and ethical way.

Repository Link

GitHub Repository : https://github.com/JaninduHasaranga/Assessment_IT_3003.git